

Routines

When to use this

Routines let you automate console interactions—perfect for demos, scripted smoke tests, or running canned data entry flows. They replay scripted keystrokes without requiring manual input.

Key types

- **Routines** – Central registry that loads and executes routines from XML definitions and tracks buffered inputs.
- **Routine** / **XmlRoutine** – Provide queue-like storage for scripted inputs and metadata such as delays.
- **RoutineInput** – Represents a single keystroke or line of text to replay.

Minimal example

```
// Load an XML routine and prime the input buffer
var routine = new XmlRoutine("./scripts/demo.xml");
Routines.InitializeRoutine(routine, routine.Name);

while (Routines.HasBuffer())
{
    RoutineInput next = Routines.Next();
    Console.WriteLine($"Replaying input: {next.Value}", ConsoleColor.DarkCyan);
    Thread.Sleep(next.Delay);
}
```

Advanced tips

- **Integration with prompts** – Register prompts with `Routines.RegisterOptions(prompt)` so scripted routines can select options by index or label. This allows automation scripts to remain resilient even if option ordering changes, and supports branching flows when the routine selects menu options based on dynamic state.
- **Recording** – Populate `Routines.UserInputs` and call `XmlRoutine.SaveInputs` to capture real sessions for later playback. Store these artefacts alongside your project to share reproducible demos with teammates, and check them into source control so they evolve with the codebase.
- **Delay handling & timing** – Respect `RoutineInput.Delay` to mimic the cadence of real users when replaying steps. Use varying delays to highlight loading states or to throttle API-driven workflows, and insert explicit waits before actions that depend on external resources (API calls, file operations) to stabilise demos.
- **Monitoring live input** – Flip `Routines.MonitorInputs` to `true` to accumulate what users type during manual runs. You can then export these

via `XmlRoutine.SaveInputs` to create new scripted journeys, or analyse the captured inputs to identify common errors in onboarding.

- **Testing** – Combine routines with CI to validate interactive flows without relying on human testers. Pair with `Consoul.RenderOptions.WriteMode = WritePlainText` to keep CI logs readable while automation runs, and use environment-specific routine folders (e.g., `./routines/ci/`) to isolate flaky experiments from stable smoke tests.
- **Dependency injection** – Build small helper classes that wrap `Routines.InitializeRoutine` so you can inject scripted input providers during tests while keeping manual entry for production. This pattern mirrors Spectre.Console's `IAnciConsole` abstraction and helps you toggle between live and scripted inputs per environment.